Python Engineering - Class 09

Modulos e imports | Guía de ejercicios

Objetivo de la clase: Separar responsabilidades entre archivos Python y comprender imports, main(), __name__ y flujo de ejecucion.

Este documento es la guía base del ejercicio. No contiene una solución completa para copiar: organiza el problema, los pasos de implementación, los valores de referencia y la validación.

Ruta

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/class_09_modules_imports/
    |-- notes/
    `-- exercices/
```

Flujo de trabajo obligatorio

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Antes de programar, redacta estas secciones. El objetivo es separar el razonamiento de la sintaxis.

EX17 - Movimiento vertical modular

Problema: Dividir la simulacion en vertical_motion.py, data_processing.py y main.py.

Entradas de referencia: Configuración y trayectoria de movimiento vertical

Ecuaciones / relación: Mismas ecuaciones Euler/procesamiento de semanas anteriores

Comprobación esperada: Los módulos importados no deben ejecutar el flujo main automáticamente; process_trajectory devuelve datos y cada módulo importa sus dependencias.

Guía de implementación

- 1. Mover fisica a vertical_motion.py.
- 2. Mover procesamiento a data_processing.py.
- 3. Importar funciones en main.py.
- 4. Definir main().
- 5. Proteger ejecucion con __name__ guard.

Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

Validación final

Responde estas preguntas como comentarios al final del ejercicio. El objetivo es demostrar que puedes explicar el flujo del código.

- Que es un modulo?
- Que ocurre al importar un archivo Python?
- Que diferencia hay entre import module y from module import function?
- Que contiene __name__ al ejecutar directamente un archivo?
- Por que process_trajectory debe devolver datos en lugar de imprimirlos?
- Que responsabilidad tiene main.py?

Criterio de cierre

La clase se considera completada cuando el código se ejecuta, las comprobaciones de referencia tienen sentido, puedes explicar cada bloque sin leer una solución y puedes modificar al menos un input anticipando su efecto.